Data Engineering & Analytics Interview Guide

Comprehensive Study Notes: Dimensional Modeling, Schemas, Warehouses, Lakes, and Scenario-Based Technical Q&A

Part 1: Conceptual & Schema Design Questions

1. What is the business and performance rationale for choosing a Star Schema over a Normalized (3NF) Snowflake Schema?

A star schema denormalizes dimensions into flat tables surrounding a central fact table. This structure drastically **minimizes the number of table joins** required to answer analytical queries, speeding up read performance for BI tools. While it introduces minor data redundancy, storage is cheap, and query execution engines perform much better with fewer join operations.

2. Scenario: Your engineering team proposes normalizing the *DimProduct* table into *DimProduct* → *DimSubcategory* → *DimCategory* to save storage space. How do you respond?

Push back against this snowflake design unless storage constraints are critical (which is rare in modern cloud warehouses). Normalizing dimensions creates extra join overhead, complicates BI reporting layer metadata, and frustrates business users writing ad-hoc queries. Modern columnar databases compress repeated attribute text efficiently anyway, rendering the storage-saving argument negligible.

3. What is a "Grain" in dimensional modeling, and why is defining it correctly critical?

The grain defines the exact meaning of a single row in a fact table (e.g., an individual line item on a sales receipt vs. an entire sales receipt summary). Getting the grain wrong breaks data integrity: if you mix grains, your metrics will suffer from double-counting or data omission during aggregations. Always define the grain *before* building tables.

Part 2: Slowly Changing Dimensions (SCD) Scenarios

4. Scenario: A retail customer changes their email address on the e-commerce app. Marketing wants historical analytics to reflect communications sent to their *current* email, but the fraud department needs to audit transactions using the email active *at the time of purchase*. Which SCD approach handles this?

SCD Type 2 (Add New Row). By creating a new version of the customer dimension record with a new surrogate key and valid date ranges (*effective_date* to *expiry_date*), historical fact records pointing to the old surrogate key preserve the old email, while new fact records link to the new surrogate key, satisfying both analytical needs.

5. When would you deliberately choose SCD Type 1 over SCD Type 2?

Use **SCD Type 1 (Overwrite)** when historical tracking is unneeded or when fixing data entry errors (e.g., correcting a misspelled customer name or fixing a typo in a product description). Type 1 simply overwrites the old value in place without creating new rows.

Part 3: Architecture & Storage (Warehouse vs. Lake vs. Lakehouse)

6. Scenario: Your company stores all raw JSON clickstream logs in an S3 data lake, but analysts complain that ad-hoc SQL queries take hours and fail randomly. How do you evolve this architecture?

Transition from a raw data lake to a **Lakehouse architecture** using open table formats like **Apache Iceberg or Delta Lake**. By applying a structured table layer over S3 files, you add ACID transactions, schema enforcement, and file optimization (compaction/partitioning), giving you data warehouse reliability with data lake flexibility.

7. Why do traditional Data Warehouses (like Snowflake or BigQuery) enforce strict ACID transactions, and why does it matter for analytics?

ACID (Atomicity, Consistency, Isolation, Durability) guarantees that concurrent data loads and analytics reads never expose partial or corrupted state changes. For financial reporting and executive dashboards, incorrect numbers due to race conditions or failed mid-stream writes are unacceptable; ACID ensures data trustworthiness.

Part 4: SQL & Implementation Challenges

8. Write a SQL snippet using a window function to find the top-selling product category by total revenue using our star schema structure.

```sql
SELECT
    p.category,
    SUM(f.total_amount) AS total_revenue,
    RANK() OVER (ORDER BY SUM(f.total_amount) DESC) as revenue_rank
FROM FactSales f
JOIN DimProduct p ON f.product_key = p.product_key
GROUP BY p.category;
```

9. Scenario: A pipeline load runs twice due to a scheduler retry, resulting in duplicate entries for *FactSales* on the same day. How do you design your pipeline to prevent this?

Implement **Idempotency** using a partition overwrite strategy or a ***MERGE*** (upsert) command. For a daily load, the pipeline should first delete all records matching that specific ***date_key*** inside a transaction before inserting the fresh batch payload, ensuring that running the pipeline 1 or 5 times leaves the database in the exact same state.

Part 5: Advanced Modeling Patterns

10. What is a "Conformed Dimension", and why is it essential for enterprise data warehousing?

A conformed dimension is a dimension table (like ***DimDate*** or ***DimCustomer***) that is shared identically across multiple fact tables or data marts. It ensures enterprise-wide consistency, allowing business units to combine metrics from different business processes (e.g., combining ***FactSales*** and ***FactSupportCalls***) using the exact same definitions.

11. What is a "Degenerate Dimension", and where does it live?

A degenerate dimension is an attribute that has no descriptive attributes of its own to form a separate dimension table, so it lives directly inside the **Fact Table**. Classic examples include transaction invoice numbers, order numbers, or receipt numbers.

12. Scenario: You are tracking user activity where a single user can belong to multiple user groups simultaneously, and groups can have multiple users. How do you model this relationship in a dimensional schema?

Use a **Bridge Table** (or junction table) placed between the ***DimCustomer*** (user) table and a ***DimUserGroup*** table. The bridge table holds pairs of foreign keys (***customer_key***, ***group_key***) alongside weighting factors if fractional allocations are required for reporting rollups.